

ADR-004: how a YARG client reaches a remote library

Status, corrected 2026-11-02 — this line said "*increments 2 and 3 are proposals*" long after increment 2 shipped:

- **Increment 1 — built, verified and finished** (`yarg` `7590443` through `5865c24f` on `dev`).
- **Increment 2 — built** (`yarg` `36ca1f2a`), **and reachable by a player since** `yarg` `55eda66d` , which replaced a `YARG_REMOTE_CATALOG=1` environment variable with a *Browse Server Library* setting. It had been complete and switched off for the whole time in between, behind a variable no player could set and nothing documented. Verified against the live 4,085-song server on 2026-11-02 rather than by reading: three songs known only from the catalogue downloaded, were scanned by the real scanner and returned charts — `Editor/RemotePlaybackProbe.cs` , cold 131–1208 ms, warm 4–9 ms.
- **Increment 3 — NOT built as specified below, and the specification is the part that aged.** What shipped gets increment 3's *browsing* benefit by a different route: entries are built from the server's JSON as `RemoteSongEntry` , so browsing 4,085 songs scans nothing. That sidesteps finding 6 entirely — no `songcache.bin` format to speak, no YARG.Core addition to ask for — which is why it could ship while the route below could not. What remains unbuilt is the part below that this does not cover: **playing** a song still downloads it and scans that one file.

Increments 2 and 3 are also **no longer blocked on anybody's answer** — see the scope note immediately below.

Scope, restated 2026-09-08

This ADR was written expecting upstream to decide its fate, and its open questions were phrased as things to ask. **That framing was wrong and is corrected here.** Jay's position, stated 2026-09-08:

> Regardless of whether the YARG devs want to incorporate our work directly or not, or if they approve of our additions in part or in whole, we are going to continue our work. I would *like* to have their approval and cooperation, but it isn't required. This is an open-source project, as is YARG, and we are fully allowed to fork, patch, or modify their code with our project so long as we are attributing them correctly and maintaining license conditions.

So upstream is a **collaborator we would welcome, not a gate we are waiting at.** Three things follow, and they are consequences rather than restatements:

1. **"Waiting on upstream" stops being a status.** Increments 2 and 3 are now ordinary engineering decisions, made on our own judgement, in the fork. What remains true is that building against a guess about their preferred shape has a cost — but the cost is a possible rewrite later, not a blocked project, and that is a trade we can price.
2. **The obligation this creates is the licence's, and it binds now rather than at release.** LGPL-3.0-or-later carries GPL-3.0 §5(a): a modified work must carry prominent notices saying it was changed and when. **The fork did not do that** — its README was upstream's, unchanged, with nothing marking it as modified. Fixed 2026-09-08: a notice at the top of the README and `FORK-NOTICE.md` recording what changed and when. Worth stating plainly that this was a real gap and not a formality we were already meeting.
3. **The question "would upstream take this?" is replaced by a better one: "what is the smallest change to THEIR code we can live with?"** Every line we add to `YARG.Core` is a line we rebase forever. That is now the design constraint, and it is a sharper one than approval was.

The patches in `docs/patches/` keep their value either way: they are correct fixes to real upstream bugs, they are what we would open a PR with if the moment comes, and until then they are what we would apply to our own copy. **Constrained by ADR-001. Informed by UPSTREAM.md.**

Measured against `yarg` at `3673672` (branch `dev`) with the `YARG.Core` submodule at `028969a`. Every file and line cited below was read on 2026-09-07; where this document says "there is none", that is a grep with a count, not an impression.

Context

Phase 2 gave us a server that hands out ordinary `.sng` files and a `yarg-sync` client that mirrors a server's library into a local folder an **unmodified** YARG then plays. That works today and is the reason this project is useful to somebody now rather than after a merge.

It also has a ceiling: the player has to hold the whole library on disk, and has to run a separate tool to get it. The Phase 3 question is what the game itself should do — and, separately, what could plausibly be upstreamed.

`docs/UPSTREAM.md` covers the political half: their `CONTRIBUTING.md` sorts every feature into six tiers and the tier decides whether a PR is read at all, the tier for a remote song source is not published, and the answer is a Discord question that has not been asked yet. This ADR is the technical half, and it deliberately does not wait for that answer — the decision below is designed so their answer changes the *destination* of the work, not the work.

What the code actually says

Six findings. They are what the decision rests on, so they are stated with citations rather than summarised.

1. There is no networking in YARG.Core at all.

```
grep -rniE "HttpClient|System\.Net|UnityWebRequest|WebRequest" YARG.Core/ --include=*.cs
0
```

Zero matches. The Unity project *does* do networking — `Assets/Script/Integration/DiscordController.cs`, `Localization/LocalizationManager.cs`, `Song/SongSources.cs` — but none of it produces a `SongEntry`. So a remote source is not "extend the existing pattern". There is no pattern.

(`yarg/docs/song_sync.md` is about audio/clock synchronisation during play. It is not prior art for song delivery, and the name is a trap.)

2. The only public scan API takes a list of local directories.

`Song/Cache/CacheHandler.cs:43`

```
public static SongCache RunScan(bool tryQuickScan, string cacheLocation, string badSongsLocation,
                                bool fullDirectoryPlaylists, List<string> baseDirectories)
```

That is the whole public surface of the scanner. `List<string>` of paths is baked in from `RunScan` down through `IniEntryGroup(dir)`, `ScanDirectory(DirectoryInfo, …)`, `ScanFile(FileInfo, …)`. There is no injectable source or provider interface to implement.

3. Exactly one seam over "where the bytes come from" already exists, and it is not enough.

`Song/Entries/Ini/SongEntry.IniBase.cs:82`

```
protected abstract FixedArray<byte>? GetChartData(string filename);
```

`LoadChart()` (`:92`) is built on it and is genuinely source-agnostic. **Everything else is not.** `SngEntry` calls `SngFile.TryLoadFromFile(_location, …)` at seven sites (`SongEntry.Sng.cs:33, 45, 73, 103, 199, 219, 241`) and touches `File.Exists` / `File.OpenRead` / `FixedArray.LoadFile` at nine more. `_location` (`SongEntry.IniBase.cs:71`) is a `string` set once in the constructor and is both `ActualLocation` and `SortBasedLocation`.

`SngFile` has exactly one loader — `TryLoadFromFile(string filename, bool loadMetadata)` (`IO/SngHandler/SngFile.cs:100`) — and it opens a `FileStream` directly. There is no stream-taking overload. `FixedArray` already has `ReadRemainder(Stream)` and `Read(Stream, long, bool)` (`IO/FixedArray/FixedArray.cs:28, 40`), so adding one is small, but it does not exist today.

4. Both ini entry classes are `internal sealed` with private constructors.

`SngEntry` (`SongEntry.Sng.cs:14`, ctor `:339`), `UnpackedIniEntry` (`SongEntry.UnpackedIni.cs:15`, ctor `:281`), and their base `IniSubEntry` (`SongEntry.IniBase.cs:37`) are all `internal`. A remote entry type **cannot** be added from outside the assembly. Any entry-level approach is a change to YARG.Core itself, not an extension of it.

5. Scanning needs the chart bytes; deserialising does not.

`IniSubEntry.ScanChart` (`SongEntry.IniBase.cs:152`) is handed the full chart file, fills `AvailableParts` through the `Midi_*_Preparser` family (`SongEntry.Scanning.cs:9`), rejects with `ScanResult.NoNotes` unless `IsValid(in parts)` (`Scanning.cs:108`), and computes `entry._hash = HashWrapper.Hash(file.ReadOnlySpan)` (`IniBase.cs:214`). That hash is the key of `SongCache.Entries` (`Song/Cache/SongCache.cs:8`).

So an entry cannot be built from metadata alone **through the scan path**. It can through the cache path: `SngEntry.ForceDeserialize` (`SongEntry.Sng.cs:377`) reads hash, parts and metadata straight out of the cache and never touches the file, where its sibling `TryDeserialize` (`:355`) calls `AbridgedFileInfo.Validate` and `SngFile.ValidateMatch` first. **That asymmetry is the closest thing in the codebase to a precedent for a server-supplied entry.**

6. The cache is versioned and stores absolute paths.

`CacheHandler.cs:38` — `private const int CACHE_VERSION = 26_09_04_00;` (a `YY_MM_DD_RR` date-revision, checked at `:943`, written at `:965`; a mismatch discards the whole cache). `IniEntryGroup.Serialize` writes the group's absolute `_directory` (`IniEntryGroup.cs:38`) and then per-entry paths relative to it (`:53`).

This is independent confirmation of a permanent non-goal already on the books: **we do not generate `songcache.bin`.** It has a version constant that moves whenever upstream feels like it, and it embeds machine-specific absolute paths.

One more, because it is load-bearing for anything that serves `.sng`: **YARG does not validate the `.sng` version field.** `SngFile.TryLoadFromFile` reads it into `sng.Version` and moves on (`SngFile.cs:120–122`); the only use is `ValidateMatch(filename, versionToMatch)` (`:144`), which compares the file against the *cached* value to detect a changed file. Our `sng.Version1` comment already says this. It is now measured.

Decision

Build the remote source in the Unity layer, as three increments, and commit now only to the first.

The shape of that is set by finding 1 and finding 4 together: YARG.Core has no networking and cannot be extended from outside, and the Unity project already does networking. So **YARG.Core stays offline and the Unity layer fetches.** A design that puts an `HttpClient` inside YARG.Core is both the larger change and the harder one to upstream, and nothing requires it.

Increment 1 — the managed mirror folder. Zero YARG.Core changes.

The game keeps a folder it owns, mirrors a server's library into it exactly as `yarg-sync` does today, and adds that folder to the scan list. Everything downstream — scanning, caching, playing, sorting — is the path YARG already walks for a folder of `.sng` files.

Upstream already does exactly this, and that is the strongest argument in this document. `SongContainer.RunRefresh` (`Assets/Script/Song/SongContainer.cs:153`) builds its directory list from the player's `SongFolders` **and then appends one more**:

```
var directories = new List<string>(SettingsManager.Settings.SongFolders);
string setlistPath = PathHelper.SetlistPath;
if (!string.IsNullOrEmpty(setlistPath) && !directories.Contains(setlistPath))
{
    directories.Add(setlistPath);
}
```

`PathHelper.SetlistPath` is not a folder the player added. `FindLauncherPaths()` (`Assets/Script/Helpers/PathHelper.cs:150`) reads the **YARC Launcher's** `settings.json`, takes its `download_location`, and derives `<that>\Setlists` — a folder populated by a separate program, delivered out of band, appended to the scan list silently, and skipped cleanly when it does not exist.

That is structurally identical to what increment 1 needs. The pattern is not new, it is not ours, and upstream wrote it. What we would be adding is a second producer for the same shape of folder — which makes this a much smaller thing to ask about than "support remote libraries", and a much smaller thing to review.

What it costs the player: the whole library on disk, and a sync before play — which is automatic since `yarg 7996c46b`, so it costs waiting rather than remembering.

Built, and measured against a real server

`yarg 7590443`, with the menu integration finished over `4c24dde6` … `113a4b8c`. `PathHelper.ServerLibraryPath` and `SongServerSync.Sync(url, dest)` do the work; the player sees a **Song Server** settings tab carrying the URL, a live reachability line, what this machine holds, a **Sync On Startup** toggle (on by default) and Sync / Cancel. `server:yes` filters the music library to what the mirror brought in. **Nothing in YARG.Core was touched**, exactly as this ADR predicted.

Two things learned finishing it, both about the client rather than the design:

- **A startup quick scan cannot see new files.** `CacheHandler.QuickScan` only deserialises `songcache.bin` and falls through to a full scan solely when it parses zero entries, so syncing before the ordinary startup scan would have fetched songs that stayed invisible until a manual refresh. The scan mode is chosen by what the sync did.
- **Reachability needs its own budget**, separate from how long a download may take, or every launch with the server off sits on a frozen loading screen. Five seconds, measured at 5.1 s against an unroutable address.

`Assets/Editor/SongServerSyncSmokeTest.cs` runs a real sync over a real network from batchmode and exits non-zero on any failure. Against the vault2 deployment:

```
server=23 had=0 downloaded=23 failed=0 unmanaged=0 bytes=238215
PASS - 23 songs mirrored and verified, second run downloaded nothing
```

238,215 bytes is byte-for-byte what four concurrent `yarg-sync` clients pulled from the same server, so the C# client and the Go one agree exactly — a cross-check neither could give on its own. Every file's chart hash is verified against the name it was served under, using YARG.Core's own `SngFile` and `HashWrapper`; that is SHA-1 over the chart bytes, precisely what the scanner computes, so a file that passes is a file the scanner will agree with. Idempotence is asserted rather than assumed, because it is the property that makes this safe to run on every launch.

It never deletes. Only `<40 hex>.sng` is ours; everything else is counted and left alone. There is no prune. The one exception is a dead `.part` from a failed download, which is swept on the next run — it is ours, it is not a song, and left alone it accumulates one per failure.

The integrity guarantee is tested, not merely written down. `Assets/Editor/HostileServerProbe.cs` serves a body cut in half mid-transfer, a real archive under someone else's hash, a 500, and random bytes. No bad archive is ever named, and the one good song still arrives after four consecutive failures. It also found two bugs in YARG.Core's own `SngFile.TryLoadFromFile` failure path — see UPSTREAM.md, which carries a `git am`-ready patch.

The constraint this ADR missed: Unity blocks plain HTTP

The first smoke test failed with `InvalidOperationException: Insecure connection not allowed`. Unity's `insecureHttpOption` defaults to `NotAllowed`, and **a song server on a LAN is plain HTTP — that is the normal case, not an edge case.** Nothing in the six findings above predicted this, because it is a Player Setting rather than an API.

Changed to `DevelopmentOnly` rather than `AlwaysAllowed`: that unblocks the editor and development builds and ships nothing weaker to players. **What a release build should do is a real open question** — relax the setting, require HTTPS from the server, or ask the player to opt in per host — and it is now a question for upstream rather than a default quietly changed in a fork.

Gaps, stated rather than discovered later

- ~~A **300 Multiple Choices** response is recorded as a failure rather than resolved.~~ **Closed in `yarg` `c87f0f5`.** The client now picks the smallest package hash ordinally, matching `yarg-sync`. The rule itself does not matter; that it is total and deterministic does — two clients choosing differently would be a difference nobody would look for until two people compared scores on what they believed was the same song. Measured against a library with a real duplicate, checking *which* package arrived rather than only that one did; see `docs/TEST-CORPUS.md`.
- `SongServerUrl` is a **hidden setting** edited in `settings.json`. There is no `AbstractSetting<string>` visual in this project except the IPv4 one, so a URL row means a new setting type *and* a new prefab. The working half landed first; the row can follow without changing anything under it.

Increment 2 — fetch a song when it is played.

> **MEASURED 2026-09-08, and the claim below is wrong.** `Editor/FetchOnPlayProbe.cs`. The correction and what replaced it are immediately after this block; the original text is kept so the mistake is legible rather than quietly edited away.

~~This is the increment that needs YARG.Core, and finding 3 says exactly how much: two seams, not a redesign.~~

- `SngFile.TryLoadFromStream(Stream, bool loadMetadata)` beside the existing file loader. `FixedArray` already reads from streams; this is plumbing, and it is useful to upstream independently of anything remote.
- A way for an entry to **materialise its bytes before a load**. The honest version of this is not a new abstraction over `_location` — that would touch sixteen call sites in `SngEntry` alone — but a single hook the load methods call first, whose default is a no-op.

An entry in this mode is a real local file by the time anything reads it, so the cache, the hash and `ValidateMatch` all keep working unchanged. The player still holds every song they have *played*, not every song in the catalog.

Not committed to. It is written down here so that the seam is chosen deliberately when the time comes, rather than discovered under pressure.

What the measurement actually found

Increment 2 needs two things to be true. Only one of them was ever in doubt, and the ADR did not separate them.

1. A place in the Unity layer to fetch from, after the song is chosen and before anything reads its bytes. There is one, and it is the same shape increment 1 already uses. `GameManager.Loading.Start` holds a `LoadingContext` and queues `LoadChart` and `LoadAudio` onto it *afterwards*; nothing between the top of that method and those two lines touches the song's file — the only use of `Song` before them is a log line reading `Name` and `Artist` from metadata already in memory. Established by reading, and stated as reading.

2. A `SongEntry` that exists while its file does not. Without this there is nothing to fetch on play: the entry only exists because the file already does, and the increment collapses into increment 1. **This was measured, and the answer is yes** — with a condition that turns out to matter more than the seam ever did.

Measured	Result
Full scan, both files present	2 entries
Delete one file, then **quick** scan	**2 entries.** The deleted song survives, `ActualLocation` still points at it, `File.Exists` there is `false`
Delete one file, then **full** scan	1 entry. It is gone

So `YARG.Core` does **not** need changing for increment 2. `QuickScan` deserialises `songcache.bin` and never walks the filesystem, so an entry outlives its bytes, and the Unity layer can put the file back before the loader ever looks.

The cost that replaces the one the ADR was worried about

Every player-facing refresh in this game is a FULL rescan. Four call sites, all passing `quick: false`: the music library's own refresh, the Song Manager settings header, a settings change that touches song folders, and our own startup path when it fetched something. A full rescan drops every entry whose file is absent.

That is worse than it sounds, because **the game's own error path tells the player to do it**: a chart that fails to load raises "*Chart requires a rescan!*" and sends them straight to the Song Manager tab. A player following the game's advice would silently lose every song they had not yet played.

So the honest verdict is not "increment 2 is cheaper than we thought". It is:

- **The `YARG.Core` seam was never the blocker.** That claim was reasoned and is now retracted.
- **Increment 2 as written is not a stable middle step.** It is increment 1 plus a cache trick, and the trick is undone by an ordinary, documented, game-suggested action.
- **The real question is where entries come from across a rescan**, and there are only two answers: files on disk (which is increment 1, working today), or the *scanner* learning about the server — which is increment 3, and which is where the `YARG.Core` conversation genuinely belongs.

Recommendation: skip increment 2. Not "defer" — skip. It buys disk savings in exchange for a catalog that evaporates on an action the game itself recommends, and the seam it was going to justify buying is actually needed one increment further along.

Increment 3 — server-supplied entries, skipping the scan entirely.

A 30,000-song server would make the client scan 30,000 downloaded charts to learn what the server already knows. The server has the hash, the parts and the metadata; `ForceDeserialize` (finding 5) proves an entry can be built from exactly that.

Deferred, and the reason is finding 6. Consuming that path means either speaking the cache's serialisation format — versioned by a constant that moves without notice, with no stability contract — or adding a parallel constructor to YARG.Core. The first is the `songcache.bin` non-goal wearing a hat. The second is a real API addition and needs upstream to want it.

Our own scale measurements say when this starts to matter: indexing is linear at ~0.57 ms per song, so 30,000 songs is ~17 s of client-side scan. Annoying, not disqualifying. Increment 3 is a performance feature, and it should not be built before increment 2 makes it observable.

What this deliberately does not do

- **No `HttpClient` in YARG.Core.** Finding 1 makes that a change of character to the library, not an addition to it.
- **No streaming audio playback over HTTP.** `LoadAudio` wants random access; serving that well is a different project. Increment 2 downloads the `.sng` and plays it locally.
- **No `songcache.bin` generation.** Unchanged permanent non-goal, now with a version constant and an absolute path to point at.
- **No CON/mogg decryption**, which upstream independently lists as out of scope.

Consequences

- **Phase 3 can start now.** Increment 1 touches only the fork's Unity layer and needs no answer from anyone.
- **There is a precedent to point at.** The Official Setlist is already delivered out of band by a separate program into a folder YARG appends to its own scan list. A remote library is a second producer for that same shape, not a new concept.
- **The upstream ask gets smaller and more concrete.** It is no longer "support remote libraries"; it is "take `SngFile.TryLoadFromStream`, and consider a materialise hook on `IniSubEntry`". Both are small, both are useful outside this feature, and neither drags networking into YARG.Core. That is a much better thing to put in a Discord question than a paragraph about song servers.
- **The library becomes manageable without the game.** That is a project goal in its own right, not a by-product: the browse UI over `/api/v1/songs` already answers a 10,000-song catalog in ~140 ms from any device on the network with YARG closed, and increment 1 means the machine that *plays* no longer has to be the machine that *holds* the library. (Upstream's party-mode requests sit near this and are deliberately not ours — see UPSTREAM.md.)
- **A player on increment 1 pays disk for simplicity.** That is the right first trade: the Pi-with-a-small-card case that increments 2 and 3 exist for is exactly the case we have never measured on real hardware.

Open questions

Reframed 2026-09-08. These were written as questions for upstream. Upstream's answers would be welcome and are not required, so they are restated here as questions for us — with the one genuine dependency named as what it is.

1. ~~**Which tier does a remote song source fall into?**~~ **No longer load-bearing.** It decided whether this was *upstreamable*, not whether it was *buildable*. The Discord post is drafted in `docs/UPSTREAM.md` and is deferred to the bottom of the queue until there is a public beta to show; nothing here waits on it.

ANSWERED for increment 2 on 2026-09-08, and the answer was no. It does not need `YARG.Core` — and measuring that turned up a bigger problem than the one it settled, which is why increment 2 is now recommended for SKIPPING rather than building. See the increment 2 section above. Increment 3 is where the `YARG.Core` conversation actually belongs.

The question as it was posed: do increments 2 and 3 actually need `YARG.Core` changes at all? This has not been measured, and the badge on 2026-09-08 is the reason to measure it rather than inherit the answer — that blocker was recorded as "needs the Unity editor" through four handoffs and turned out to be false of the feature, only true of one implementation of it. Increment 2 is *fetch a song when it is played*. If the Unity layer can materialise the file into the mirror folder **before** the loader runs, the loader never knows the difference and `YARG.Core` is untouched — exactly the trick increment 1 already pulls. That is a measurement, not an opinion, and it decides whether increment 2 costs a submodule fork or an afternoon.

If it does need `YARG.Core`: the submodule points at upstream (`https://github.com/YARC-Official/YARG.Core`, pinned at `028969a9`). Modifying it means forking it to Vault2 GitLab, repointing the submodule, and carrying our patches across every upstream update forever. That is the real price of increment 2, it is payable, and it should be paid deliberately rather than discovered halfway through. 2. **Would they take `SngFile.TryLoadFromStream` on its own?** It is a small, general improvement with no remote-library baggage. If the answer is yes, that is worth doing first regardless of what happens to the rest. **Nothing is in flight on it** — searched 2026-09-07: the open PRs on `YARG.Core` are vocals (#437) and chart parsing (#204), and nothing has touched `SngFile` since the SNG handling changes of January 2024. 3. **Is anyone upstream already on this?** ~~Not searched exhaustively.~~ **Answered 2026-09-07, and the answer changed two things in this document's neighbourhood.**

The remote song source itself is unclaimed. No issue and no PR proposes fetching songs from a server.

The control channel is not. #984 — open as a draft since February 2025 — puts an HTTP server *inside* YARG on port 9090, lists the library, and selects a song in the running game when you click it. Its author calls it "*Very basic version of #860*". A contributor tried it, said "*it works really well*", and asked for it to be reshaped into a JSON API on `/api` with a bundled HTML frontend — GET for the song list, POST to change the selection. It then stalled with merge conflicts and no reviews.

Two consequences for this ADR. First, **the decision above is reinforced rather than threatened**: finding 1 argued the Unity layer should do the networking because YARG.Core has none, which was an inference; #984 is a Unity-layer HTTP server that upstream reacted to with enthusiasm, which is evidence. Second, **#984 is the opposite direction from increments 1–3 and does not overlap them** — it is the game serving control, not the game fetching content — so the two compose rather than compete, and `docs/UPSTREAM.md` now asks upstream whether they consider them one conversation or two.

Also merged since: #1540, a static "Web Page" export of the library as a self-contained HTML browser. Not live, cannot select a song, not the party-mode frontend — but YARG now ships an HTML song browser, which is prior art for the one #984's discussion asks for.

1. **What does the managed mirror folder do about songs the player deleted by hand?** `yarg-sync` already treats anything that is not `<40 hex>.sng` as the player's own and never touches it. The in-game version inherits that rule and should say so where a player can read it.
